InspectAI

Industrial visual anomaly detection - project summary

A research-oriented Python package for MVTec-style quality inspection. Long-term goal: a defensible portfolio system that can later support real-time inference. This PDF describes architecture, milestones, and planned deep-learning parts. No model has been trained yet. Dataset numbers below are EDA facts, not experiment scores.

Current status (done)

? Python 3.11 src-layout package (inspectai) with pytest and Ruff.

? Milestone 1: transistor EDA notebook; data/raw/ is read-only.

? Milestone 2: framework-independent MVTecDataset (NumPy/Pillow, no PyTorch).

? Train = normals only; test discovers defect folders; masks uint8 {0,1}; native 1024x1024.

? Unit tests on fake trees plus transistor integration tests (skip if data missing).

Immediate next step

Milestone 3 - PyTorch adapter and experiment contract (not modeling yet).

? Install PyTorch as an explicit, approved dependency.

? Add a thin torch.utils.data.Dataset wrapper around MVTecDataset (CHW tensors).

? Do not introduce aggressive resize that would hide small transistor lead defects.

? Write hypothesis, baseline, and metrics before any training run (AGENTS.md).

? The first model experiment is Milestone 4, after the adapter lands.

Repository parts

src/inspectai/data/
MVTec loading, pairing, validation, paired-transform hook. Implemented.

src/inspectai/models/
PyTorch modules. Empty. Planned AE / PatchCore backbone / later detectors.

src/inspectai/evaluation/
Image/pixel AUROC, PRO, thresholds, pass/fail. Planned.

src/inspectai/visualization/
Overlays and score maps. Helpers still live in the EDA notebook.

src/inspectai/inference/
Single-image, batch, later stream scoring. Planned.

src/inspectai/utils/
Seed, paths, logging. Minimal later.

configs/
data / model / experiment YAML. Placeholders only.

notebooks/
EDA and analysis only. 01_mvtec_eda.ipynb exists.

scripts/
Thin CLIs (train, eval, infer) calling the package. Planned.

tests/
Package smoke plus MVTec unit/integration. Implemented.

data/raw/
Immutable MVTec AD. Transistor: 213 train good, 60 test good, 40 anomalous.

outputs/
figures, checkpoints, experiment logs (gitignored contents).

Roadmap to a complete portfolio project

Stage	State	Scope
M1 EDA	**DONE**	Understand transistor: imbalance, tiny leads, 1024px, mask pairing.
M2 Data API	**DONE**	MVTecDataset + MVTecSample; split-specific validation; tests.
M3 Torch adapter	**NEXT**	Install torch; Dataset wrapper; DataLoader smoke; no training.
M4 Baseline AD	**PLANNED**	Hypothesis + metrics first. Frozen encoder + kNN/coreset or similar.
M5 Strong AD	**PLANNED**	Memory-bank method (PatchCore-style) at high resolution or patches.
M6 Second family	**PLANNED**	Reconstruction autoencoder for comparison, not as the only model.
M7 Evaluation	**PLANNED**	Image AUROC, pixel AUROC, PRO; per-defect tables; no invented scores.
M8 Categories	**PLANNED**	Same pipeline on more MVTec classes; configs per experiment.
M9 Inference CLI	**PLANNED**	scripts/infer.py: image in, score + overlay out.
M10 Optional product	**LATER**	Stream/API/dashboard only after models and eval are documented.

Definition of done (core project)

Reproducible train/eval on transistor; at least two model families compared with the same metrics; written limitations (small defects, resize risk); documented inference path. A real-time UI is extra, not required for the research portfolio core.

Planned models and deep-learning parts

None of the following are implemented. They are the intended stack after Milestone 3.

Embedding / memory-bank (primary AD path)

Wide-ResNet or similar frozen ImageNet backbone, patch-level memory (PatchCore-style). Fits MVTec one-class training and small defects if patches stay high-resolution. Typically little or no gradient training; scoring is nearest-neighbor distance in feature space.

Reconstruction (comparison baseline)

Convolutional autoencoder trained on good images only. Residual maps as anomaly scores. Useful as a baseline; often weaker on fine lead defects than patch embeddings.

Self-supervised features (optional, after baseline)

DINO / ViT features instead of a supervised CNN, same memory-bank head. Only after a working PatchCore-style baseline exists.

Detection (optional, not the default)

YOLO-style detectors need box labels. MVTec AD is one-class segmentation. Use YOLO only if the problem shifts to boxed detection, as a separate experiment.

DL system pieces (engineering, not architectures)

? Data: MVTecDataset -> later tensor Dataset -> DataLoader. Paired (image, mask) transforms.

? Train: good-only; no defect labels in the one-class setting.

? Score: distance to memory bank, or reconstruction error (AE path).

? Eval: image AUROC; pixel AUROC / PRO vs binary masks; per-defect breakdown.

? Inference: anomaly map + scalar; threshold pass/fail; overlay visualization.

? Ops: experiment YAML, seeds, checkpoints under outputs/; never invent metrics.

Transistor constraints that shape the DL design

? One-class: 213 train good, 60 test good, 10 images x 4 defect types (40 anomalous).

? All RGB images and masks 1024x1024 PNG; masks pair 1:1 with defects.

? bent_lead / cut_lead can be well under 2% of pixels; 224px resize can erase them.

? misplaced is pose of the whole part (~40% of pixels), not a local nick.

? Good leads already curve into the board; bent_lead is extreme displacement.

? Default: native resolution or high-res patches until an experiment justifies resize.

Experiment contract (before any training)

? Hypothesis, e.g. patch embeddings at native scale beat an autoencoder on lead defects.

? Baseline: simple frozen-encoder kNN or AE, not a random score.

? Metrics: image AUROC overall and per defect; pixel AUROC; qualitative overlays.

? Split: official MVTec train/test only; never mix test defects into train.

? Report failures, especially tiny leads vs large misplaced regions.

Out of scope until explicitly approved

? Training loops, PatchCore/DINO/YOLO implementations, APIs, and dashboards.

? Any modification of files under data/raw/.

? Reusable training logic inside notebooks.

Next action: approve Milestone 3 (PyTorch adapter only), then define the M4 hypothesis.